Wirtualna i rozszerzona rzeczywistość

Wykład 3

Geometria, kalibracja i rejestracja

dr inż. Mateusz Pomianek

Automatyka i Robotyka, studia II stopnia, 3 ECTS

Katedra Informatyki i Automatyki

Politechnika Rzeszowska im. Ignacego Łukasiewicza

Katedra Informatyki i Automatyki Politechniki Rzeszowskiej

Rok akademicki 2026/2027

Wersja 1.3.2 14.10.2026

Plan i rytm wykładu

1. Powtórka i cele
2. Od współrzędnych do poprawnej nakładki
3. Układy współrzędnych i reprezentacja pozy
4. Diagnostyka transformacji
5. Przerwa
6. Kamera, kalibracja i rejestracja
7. Transfer do praktyki
8. Ciekawostka
9. Podsumowanie

Jedna przerwa rozdziela reprezentację pozy od modelu kamery i rejestracji.

Punkt startowy

Z poprzednich zajęć potrzebujemy trzech idei:

- wiarygodność XR zależy od zgodności bodźców i opóźnienia,
- parametr ma sens tylko wtedy, gdy znamy jego definicję i układ odniesienia,
- śledzenie 6DoF musi dostarczać zarówno orientację, jak i pozycję.

Dziś

Zapiszemy te zależności językiem geometrii: układami współrzędnych, transformacjami, kwaternionami i modelem kamery.

Po tym wykładzie potrafisz...

W3 opisywać 3DoF i 6DoF za pomocą obrotu R i translacji t;

W3 rozróżniać układy lewo- i prawoskrętne oraz jawnie konwertować dane Unity–ROS 2;

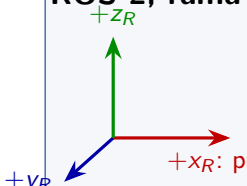
W3 wyjaśniać blokadę przegubów Kardana (gimbal lock) i rolę kwaternionów;

W3 opisywać model kamery otworkowej (pinhole camera model), parametry wewnętrzne i zewnętrzne;

W3 diagnozować błąd rejestracji z użyciem błędu reprojekcji, FRE i TRE.

Ten sam ruch robota w ROS 2 i w Unity

ROS 2, rama `base_link`

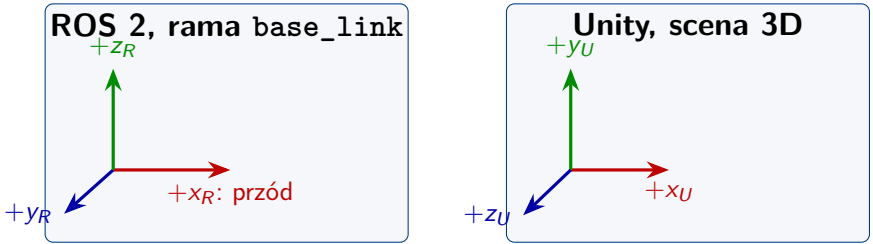


Unity, scena 3D

Reprodukowalny test

Polecenie przemieszczenia o 1 m w przód w ramie ROS: $\mathbf{p}_R = (1, 0, 0)\,\mathrm{m} \;\Rightarrow\; \mathbf{p}_U = (0, 0, 1)\,\mathrm{m}.$

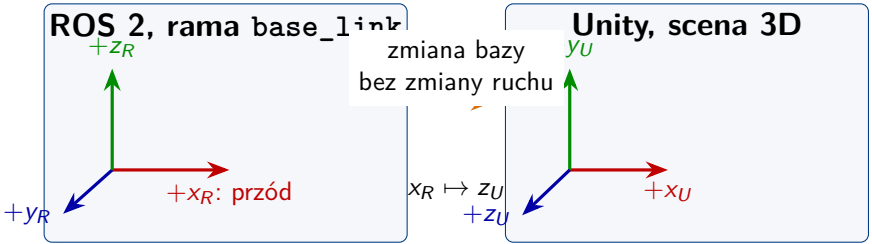
Ten sam ruch robota w ROS 2 i w Unity



Reprodukowalny test

Polecenie przemieszczenia o 1 m w przód w ramie ROS: $\mathbf{p}_R = (1, 0, 0)\,\mathrm{m} \;\Rightarrow\; \mathbf{p}_U = (0, 0, 1)\,\mathrm{m}.$

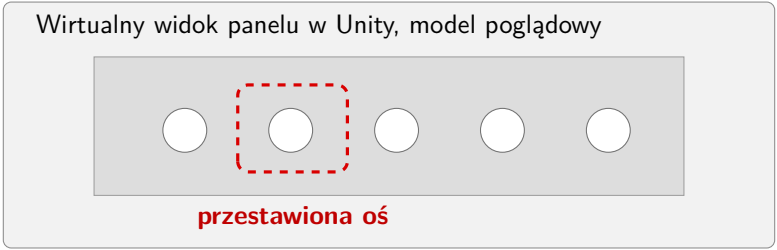
Ten sam ruch robota w ROS 2 i w Unity



Reprodukowalny test

Polecenie przemieszczenia o 1 m w przód w ramie ROS: $\mathbf{p}_R = (1, 0, 0)\,\mathrm{m} \;\Rightarrow\; \mathbf{p}_U = (0, 0, 1)\,\mathrm{m}$.

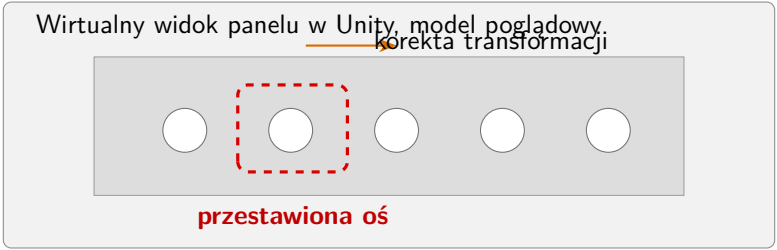
Nakładka AR: błąd orientacji widać na obiekcie



Kryterium odbioru

Najpierw porównaj wektory bazowe i znaną pozę, następnie zmierz błąd na fizycznym celu.

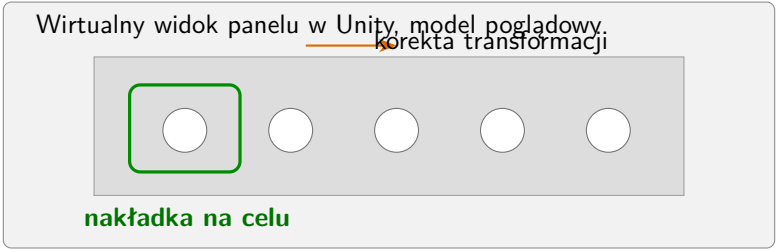
Nakładka AR: błąd orientacji widać na obiekcie



Kryterium odbioru

Najpierw porównaj wektory bazowe i znaną pozę, następnie zmierz błąd na fizycznym celu.

Nakładka AR: błąd orientacji widać na obiekcie

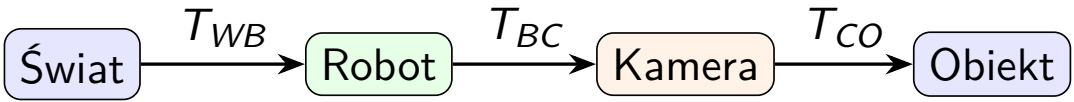


Kryterium odbioru

Najpierw porównaj wektory bazowe i znaną pozę, następnie zmierz błąd na fizycznym celu.

Układ odniesienia (reference frame)

Ten sam zapis (x, y, z) może oznaczać inny punkt w układzie świata, robota, kamery lub obiektu.



Skrętność układu jest częścią danych

Prawoskrętny

ROS domyślnie stosuje konwencję prawoskrętną [1].

Lewoskrętny

Unity stosuje układ lewoskrętny [2].

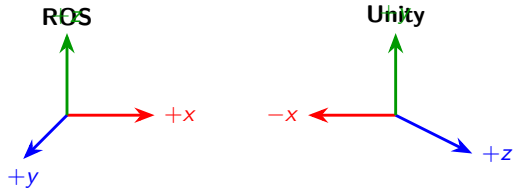
Konsekwencja

Nie wystarczy przepisać nazw osi. Trzeba jawnie przeliczyć konwencję.

Unity i ROS 2: wyprowadzamy mapowanie osi

Cecha	Unity	ROS, REP-103
skrętność	lewoskrętna	prawoskrętna
do przodu	$+z$	$+x$
do góry	$+y$	$+z$
w lewo	$-x$	$+y$

$$(x_U, y_U, z_U) = (-y_R, z_R, x_R)$$



Układ optyczny kamery (OpenCV / ROS optical): $+x$ w prawo, $+y$ w dół, $+z$ w przód. Dla ROS `base_link` w konwencji FLU:

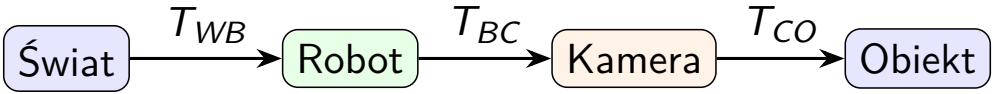
$$(x_R, y_R, z_R) = (z_o, -x_o, -y_o).$$

Macierz jednorodna łączy obrót i przesunięcie

$$T = \begin{bmatrix} R_{3\times 3} & t_{3\times 1} \\ 0\ 0\ 0 & 1 \end{bmatrix}$$

- R opisuje orientację,
- t opisuje przesunięcie,
- współrzędne jednorodne pozwalają składać oba działania jednym mnożeniem.

$$^{W}T_{E} = {}^{W}T_{B}\,{}^{B}T_{L}\,{}^{L}T_{E}$$



Nieprzemienność

W ogólności $AB \neq BA$.

Odwracanie transformacji i kierunek ramek

$$ {}^{A}T_{B} = \begin{bmatrix} R & t \\ 0\ 0\ 0 & 1 \end{bmatrix}, \qquad {}^{B}T_{A} = \begin{bmatrix} R^{\mathsf{T}} & -R^{\mathsf{T}} t \\ 0\ 0\ 0 & 1 \end{bmatrix}. $$

- Notacja ${}^{A}T_{B}$: punkt zapisany w ramce B przeliczamy na współrzędne ramki A.
- $R^{\mathsf{T}} R = I$, $\det R = +1$ dla $R \in SO(3)$.
- W ROS 2 biblioteka `tf2` zarządza drzewem transformacji; REP-105 definiuje m.in. `map`, `odom`, `base_link`.

Kolejność obrotu i translacji: przykład liczbowy

$$R = R_z(90^\circ), \quad t = (1, 0, 0), \quad p = (1, 0, 0).$$

Obrót, potem translacja

$$Rp + t = (1, 1, 0).$$

Translacja, potem obrót

$$R(p + t) = (0, 2, 0).$$

Konwersja pozy ROS–Unity: nie tylko pozycja

$$M = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \quad p_U = M p_R, \quad R_U = M R_R M^{\mathsf{T}}.$$

- $\det M = -1$: to zmiana konwencji osi, a nie macierz obrotu z $SO(3)$.
- Dla kwaternionów $q = (x, y, z, w)$: $q_U = (y_R, -z_R, -x_R, w_R)$, z dokładnością do równoważnego znaku całego kwaternionu.
- Przykład: $p_R = (0{,}4; 0{,}1; 0{,}8)$ m odpowiada $p_U = (-0{,}1; 0{,}8; 0{,}4)$ m.

3DoF i 6DoF w tym samym modelu

3DoF

Śledzona jest orientacja $R(t)$.

6DoF

Śledzone są orientacja $R(t)$ i pozycja $t(t)$.

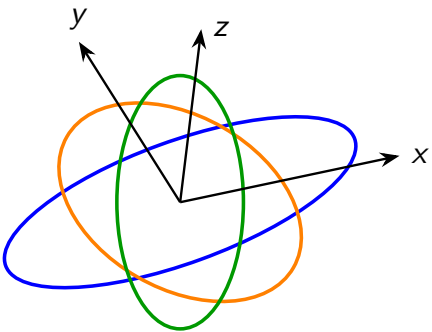
Cztery reprezentacje orientacji

Reprezentacja	**Zaleta**	**Ograniczenie**
macierz 3×3	łatwe składanie	nadmiarowe 9 liczb
kąty Eulera	intuicyjne	kolejność i osobliwości
oś–kąt (rvec)	naturalny dla PnP	niejednoznaczny dla pełnych obrotów
kwaternion	zwarty, interpolowalny	mniej intuicyjny

Gimbal lock: utrata niezależnej osi obrotu

Mechanizm

Dla pewnych konfiguracji kątów Eulera dwie osie obrotu stają się współliniowe i reprezentacja lokalnie traci jeden niezależny kierunek zmiany.



Przy pitch $= 90^\circ$ dwie osie obrotu pokrywają się

Kwaternion opisuje obrót bez sekwencji trzech kątów

$$q = w + xi + yj + zk, \qquad \|q\| = 1$$

$$q = \left(\cos\frac{\theta}{2}, \hat{\mathbf{u}}\sin\frac{\theta}{2}\right)$$

Dlaczego bez gimbal lock?

Jednostkowy kwaternion ma cztery składowe z warunkiem normy jeden; q i $-q$ opisują ten sam obrót. Nie używa trzech kolejnych kątów Eulera.

Kod: transformacja punktu między układami

```python
"""Apply a 4x4 homogeneous transform to a 3D point."""


def transform_point(T, point):
    x, y, z = point
    p = [x, y, z, 1.0]
    result = [sum(T[row][col] * p[col] for col in range(4))
     for row in range(4)]
    return tuple(result[:3])


T_world_camera = [
    [1.0, 0.0, 0.0, 0.50],
    [0.0, 1.0, 0.0, 0.10],
    [0.0, 0.0, 1.0, 1.20],
    [0.0, 0.0, 0.0, 1.00],
]

camera_point = (0.2, -0.1, 0.8)
print(transform_point(T_world_camera, camera_point))
```

Wejście: punkt 3D i macierz 4×4.

Współrzędna jednorodna $w = 1$ pozwala w jednej operacji połączyć obrót i translację.

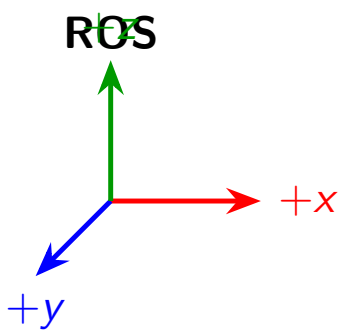
W XR: dokładnie taki krok wykonujemy przy przejściu np. z układu kamery do świata.

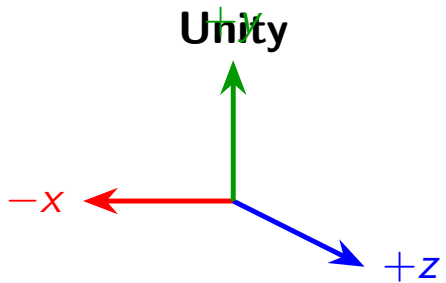
Dane ROS

Przemieszczenie $\Delta p_R = (0{,}20; 0; 0)$ m, oś $+x_R$ oznacza ruch naprzód.

Wynik Unity

$\Delta p_U = (0; 0; 0{,}20)$ m. Ruch trafia na oś $+z_U$, a nie $+x_U$.





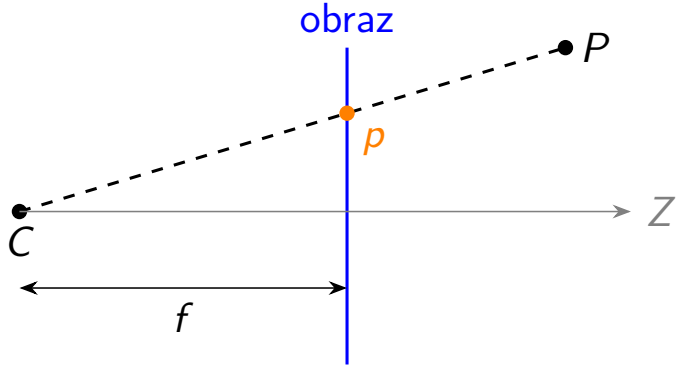
$$e_{x,R} \mapsto e_{z,U}, \quad e_{y,R} \mapsto -e_{x,U}, \quad e_{z,R} \mapsto e_{y,U}.$$

Test orientacji

Dla każdego sprawdzanego obrotu R_R wymagaj zgodności $R_U = M R_R M^{\mathsf{T}}$, ortogonalności R_U i $\det R_U = +1$.

Przerwa 5 min

Model kamery otworkowej: 3D staje się 2D



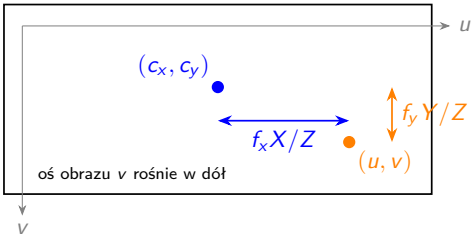
Model kamery otworkowej

Promień przechodzący przez środek projekcji wyznacza punkt obrazu.

Macierz kamery: ogniskowa i punkt główny

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- f_x, f_y: ogniskowe w pikselach,
- c_x, c_y: punkt główny,
- osobno opisujemy zniekształcenia optyczne.



Parametry zewnętrzne mówią, gdzie jest kamera

$$P_C = RP_W + t$$

Pułapka

„Kamera w świecie” i transformacja „świat do kamery” mają przeciwne kierunki.

Pełne równanie projekcji

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Czytamy od prawej

punkt świata $\rightarrow$ układ kamery $\rightarrow$ parametry wewnętrzne $\rightarrow$ piksel.

Od pozy kamery do stabilnej nakładki AR

$$V = ({}^{W}T_{C})^{-1}, \qquad \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}.$$

- Macierz widoku V jest odwrotnością pozy kamery w świecie.
- Projekcja musi odpowiadać rzeczywistym f_x, f_y, c_x, c_y i rozmiarowi obrazu.
- OpenCV używa osi obrazu y w dół. Przejście do renderera wymaga jawnej zmiany konwencji.
- Równie ważny jest czas pozy: niewielki błąd statyczny nie usuwa opóźnienia.

Przykład liczbowy projekcji kamery

$$f_x = f_y = 800 \text{ px}, \quad (c_x, c_y) = (640, 360), \quad (X_c, Y_c, Z_c) = (0{,}3, -0{,}15, 2) \text{ m}.$$

$$u = f_x \frac{X_c}{Z_c} + c_x = 760, \qquad v = f_y \frac{Y_c}{Z_c} + c_y = 300.$$

Interpretacja

Współrzędna Z_c odpowiada skali perspektywicznej. Zniekształcenia obiektywu uwzględniamy w modelu przed przemnożeniem przez K.

FRE, TRE i opóźnienie: trzy odrębne błędy

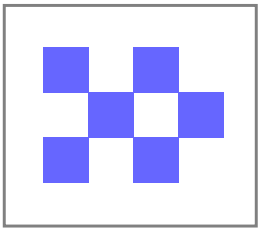
$$\mathrm{FRE} = \sqrt{\frac{1}{N}\sum_{i} \|R p_i + t - q_i\|^2}, \qquad e_{\mathrm{rot}} \approx L \tan(\delta\theta).$$

- FRE opisuje reszty na punktach dopasowania; TRE dotyczy niezależnego punktu docelowego.
- Błąd kątowy $0{,}5^\circ$: ok. $4{,}4$ mm przy $L = 0{,}5$ m i 26 mm przy $L = 3$ m.
- Błąd dynamiczny: $\delta\theta \approx \omega\tau$. Przy $100^\circ/\mathrm{s}$ i 20 ms otrzymujemy 2°.

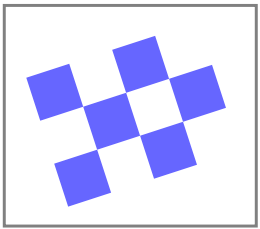
Kalibracja: zróżnicowane ułożenia wzorca

Typowy przebieg:

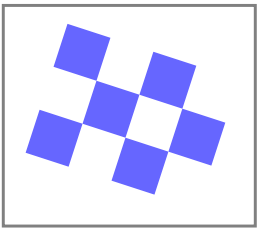
1. rejestrujemy wzorzec w różnych położeniach,
2. wykrywamy punkty,
3. tworzymy korespondencje 3D–2D,
4. estymujemy parametry,
5. analizujemy błąd reprojekcji [3].



centralnie



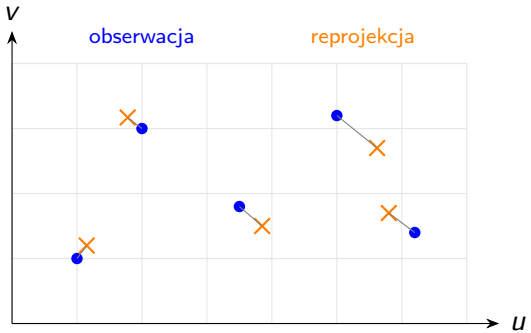
ukośnie



przy brzegu

Błąd reprojekcji: rozkład błędów w pikselach

$$e_i = \|p_i - \hat{p}_i\|_2$$



Dobra praktyka

Sprawdź medianę, RMS i położenie dużych błędów.

Problem perspektywiczny n-punktowy (Perspective-n-Point, PnP)

Znając punkty 3D, odpowiadające im punkty 2D oraz model kamery, estymujemy obrót i translację obiektu względem kamery [4].

Obserwacje odstające

Przy błędnych korespondencjach warto stosować odporną estymację, np. RANSAC [5].

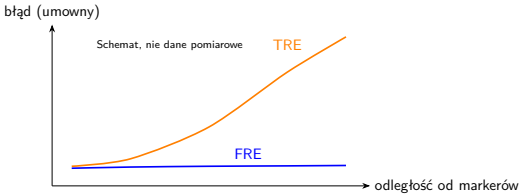
Rejestracja: niski błąd na markerach nie wystarcza

FRE

Błąd w punktach odniesienia użytych do wyznaczenia transformacji.

TRE

Błąd w punkcie, który naprawdę interesuje użytkownika.



Wniosek

Niski FRE nie gwarantuje niskiego TRE [6].



Kolejność

Najpierw konwencje osi, jednostki i ramki, następnie projekcja kamery i błąd rejestracji w obszarze pracy.

Minimalna procedura diagnostyczna

1. zamroź scenę i wybierz jedną znaną pozę;
2. sprawdź jednostki oraz definicję ramek;
3. zweryfikuj mapowanie trzech osi i znaków;
4. sprawdź kolejność oraz kierunek łańcucha transformacji;
5. porównaj przewidywany i obserwowany punkt w obrazie;
6. zmierz błąd w obszarze docelowym, nie tylko na markerach.

Test jednostkowy konwersji osi jest częścią bezpieczeństwa

W systemie robotycznym

Błędna konwersja pozycji lub orientacji może prowadzić nie tylko do złej wizualizacji, lecz także do błędnej interpretacji kierunku polecenia.

Zasada

Warstwa wizualizacji i warstwa sterowania muszą mieć jawnie zdefiniowane konwencje ramek oraz osobne testy. Nie wysyłaj poleceń do robota fizycznego na podstawie niezweryfikowanej transformacji.

Kod Python: ROS body do Unity

```python
"""Deterministic ROS body (FLU) to Unity (RUF) position mapping."""


def ros_body_to_unity(position_m):
    x_forward, y_left, z_up = position_m
    return (-y_left, z_up, x_forward)


if __name__ == "__main__":
    assert ros_body_to_unity((1.0, 0.0, 0.0)) == (0.0, 0.0, 1.0)
    assert ros_body_to_unity((0.0, 1.0, 0.0)) == (-1.0, 0.0, 0.0)
    assert ros_body_to_unity((0.0, 0.0, 1.0)) == (0.0, 1.0, 0.0)
    print("Trzy wektory bazowe: OK")
```

Zadanie
Ruch o 1 m wzdłuż osi przodu robota musi stać się ruchem wzdłuż $+z$ w Unity.

Trzy asercje
Sprawdzają wszystkie osie bazowe.

Granica
Przykład dotyczy położenia, nie orientacji ani położenia czujnika.

Unity: potwierdzenie osi na rzeczywistym widoku sceny



Miejsce na rzeczywisty zrzut ekranu: Unity 6000.6, Scene view: osie XYZ, obiekt testowy i Inspector z `Transform.position=(0,0,1)` po przemieszczeniu o 1 m wzdłuż osi przodu ROS.

Ciekawostka: Zmiana skrętności nie jest obrotem

Dlaczego to jest interesujące?

Macierz mapująca standardowe osie ROS na osie Unity ma wyznacznik -1. To sygnatura odbicia, a nie właściwej rotacji z grupy $SO(3)$. Dlatego nie wolno bezmyślnie potraktować tej samej macierzy jako zwykłego obrotu kwaternionowego. Konwersję orientacji trzeba wyprowadzić dla przyjętych baz albo powierzyć sprawdzonej bibliotece.

Co warto zapamiętać?

1. Punkt bez nazwanego układu odniesienia jest niepełną informacją.
2. Unity i ROS stosują różne konwencje osi, więc integracja wymaga jawnej konwersji i testów.
3. Macierz jednorodna łączy obrót i przesunięcie, a kolejność składania transformacji ma znaczenie.
4. Kwaterniony unikają osobliwości kątów Eulera i nadają się do płynnej interpolacji orientacji.
5. Jakość rejestracji trzeba oceniać w obszarze zadania, nie tylko przez błąd kalibracyjny lub FRE.

Diagnostyka: najpierw osie, potem kalibracja

Test rozstrzygający

Przekształć trzy wektory bazowe oraz znaną pozę. Jeśli występuje stała zamiana osi lub znaku, napraw konwencję ramek. Dopiero potem badaj reszty reprojekcji w różnych obszarach obrazu.

Przykładowe pytania egzaminacyjne

1. Jak zapisać transformację punktu pomiędzy układem kamery a układem świata za pomocą macierzy jednorodnej?
2. Jak rozpoznać i skorygować niezgodność konwencji osi pomiędzy ROS a Unity?
3. Dlaczego kolejność składania obrotu i translacji wpływa na wynik transformacji?
4. Jak odróżnić błąd kalibracji kamery od błędu rejestracji nakładki AR w obszarze zadania?

Bibliografia

[1] T. Foote i M. Purvis, „REP 103: Standard Units of Measure and Coordinate Conventions”, ROS. (), adr.: https://ros.org/reps/rep-0103.html (term. wiz. 10. 09. 2026).

[2] Unity Technologies, „Rotation and orientation in Unity”, (), adr.: https://docs.unity3d.com/Manual/QuaternionAndEulerRotationsInUnity.html (term. wiz. 10. 09. 2026).

[3] OpenCV, „Camera Calibration and 3D Reconstruction”, (), adr.: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html (term. wiz. 10. 09. 2026).

[4] OpenCV, „Perspective-n-Point Pose Computation”, (), adr.: https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html (term. wiz. 10. 09. 2026).

[5] M. A. Fischler i R. C. Bolles, „Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography”, *Communications of the ACM*, t. 24, nr. 6, s. 381–395, 1981.

[6] J. M. Fitzpatrick i J. B. West, „The Distribution of Target Registration Error in Rigid-Body Point-Based Registration”, *IEEE Transactions on Medical Imaging*, t. 20, nr. 9, s. 917–927, 2001.

Przejdź do kolejnych materiałów

← Wykład 2	Laboratorium 3	Wykład 4 →

Otwórz wybrany materiał w przeglądarce lub czytniku PDF.